

Guia de Testing Swagger

Microservicios Hotel Transilvania

Documento tecnico y practico para probar manualmente endpoints desde Swagger. Elaborado revisando controllers, services, repositories, modelos, DTOs, WebClient, SecurityConfig y GlobalExceptionHandler del codigo local.

Indice

- 1. Resumen general de microservicios
- 2. Preparacion: orden de levantamiento y uso de JWT en Swagger
- 3. Orden logico para crear datos de prueba
- 4. Guia por microservicio con endpoints, ejemplos y validaciones
- 5. Modelos y entidades por microservicio
- 6. Datos necesarios para poblar cada microservicio
- 7. Matriz de pruebas manuales
- 8. Errores esperados y pruebas negativas

1. Resumen General

Microservicio	Puerto	Ruta base	Swagger URL	Token	Dependencias
ms-auth-usuarios-service	8081	/api/v1/auth, /api/v1/usuarios	http://localhost:8081/swagger-ui.html	Parcial. /auth/login y /auth/register son publicos; usuarios requiere JWT.	No consume otros microservicios.
ms-cliente-service	8082	/api/v1/clientes	http://localhost:8082/swagger-ui.html	Si. Todos los endpoints de negocio requieren JWT y @PreAuthorize ADMIN.	No consume otros microservicios.
ms-hotel-service	8083	/api/v1/hoteles	http://localhost:8083/swagger-ui.html	GET publico; POST, PUT y DELETE requieren JWT ADMIN.	No consume otros microservicios.
ms-habitacion-service	8084	/api/v1/habitaciones	http://localhost:8084/swagger-ui.html	GET protegido por SecurityConfig general aunque el controller no tenga @PreAuthorize; POST, PUT y DELETE requieren ADMIN.	WebClient a ms-hotel-service para validar idHotel.
ms-disponibilidad-service	8085	/api/v1/disponibilidades	http://localhost:8085/swagger-ui.html	Si. GET/PUT USER o ADMIN; POST/DELETE ADMIN.	WebClient a ms-habitacion-service para validar idHabitacion.
ms-reserva-service	8086	/api/v1/reservas	http://localhost:8086/swagger-ui.html	Si. GET lista/PUT/DELETE ADMIN; GET id, POST y cambio estado USER o ADMIN.	WebClient a cliente, auth/usuarios, hotel, habitacion y disponibilidad.
ms-pago-service	8087	/api/v1/pagos	http://localhost:8087/swagger-ui.html	Si. Listar y filtros por estado ADMIN; crear y consultar por id/reserva USER o ADMIN.	WebClient a ms-reserva-service y ms-auth-usuarios-service.
ms-notificacion-service	8088	/api/v1/notificaciones	http://localhost:8088/swagger-ui.html	Si. Listar/crear/actualizar/eliminar ADMIN; buscar por id USER o ADMIN.	WebClient a cliente, auth/usuarios y reserva.
ms-resena-service	8089	/api/v1/resenas	http://localhost:8089/swagger-ui.html	Si. Casi todo USER o ADMIN; filtro por estado ADMIN.	WebClient a cliente, hotel y habitacion. No valida reserva aunque idReserva es obligatorio.
ms-servicio-adicional-service	8090	/api/v1/servicios-adicionales	http://localhost:8090/swagger-ui.html	Si. GET USER o ADMIN; crear/actualizar/eliminar ADMIN.	WebClient a hotel y reserva.

- ms-comentario-service: no existe en el proyecto revisado.
- ms-resena-service: existe y cubre reseñas/comentarios de experiencia.
- Gateway existente: api-gateway en puerto 8080, enruta paths /api/v1/** hacia los servicios registrados en Eureka.
- Swagger por servicio queda en http://localhost:{puerto}/swagger-ui.html. El gateway no expone un Swagger agregado en el código revisado.

2. Preparacion Para Swagger

Orden recomendado para levantar

- 1. eureka-server en puerto 8761.
- 2. api-gateway en puerto 8080.
- 3. ms-auth-usuarios-service en puerto 8081.
- 4. ms-hotel-service en puerto 8083.
- 5. ms-cliente-service en puerto 8082.
- 6. ms-habitacion-service en puerto 8084.
- 7. ms-disponibilidad-service en puerto 8085.
- 8. ms-reserva-service en puerto 8086.
- 9. ms-pago-service en puerto 8087.

- 10. ms-notificacion-service, ms-resena-service y ms-servicio-adicional-service.

Como obtener y usar JWT

- Crear usuario publico con POST /api/v1/auth/register. Este endpoint crea rol USER.
- Para rol ADMIN, usar POST /api/v1/usuarios con token ADMIN ya existente o sembrar un usuario ADMIN en base de datos.
- Hacer POST /api/v1/auth/login con nombre y password.
- Copiar el campo token.
- En Swagger, presionar Authorize y pegar el valor como Bearer si el cuadro no lo agrega automaticamente.

```
POST http://localhost:8081/api/v1/auth/login
{
  "nombre": "admin",
  "password": "admin123"
}
```

Respuesta esperada:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "nombre": "admin",
  "rol": "ADMIN"
}
```

3. Orden Logico Para Datos

- 1. Crear usuario/admin y obtener token.
- 2. Crear hotel.
- 3. Crear cliente.
- 4. Crear habitacion asociada al hotel.
- 5. Crear disponibilidad para cada fecha de la estadia.
- 6. Crear reserva PENDIENTE.
- 7. Confirmar reserva con PUT /api/v1/reservas/{id}/estado?estadoReserva=CONFIRMADA o crear pago APROBADO.
- 8. Verificar que disponibilidad cambie a OCUPADA.
- 9. Cancelar o cambiar una reserva confirmada y verificar que disponibilidad vuelva a DISPONIBLE.

ms-auth-usuarios-service

Campo	Valor
Puerto	8081
Ruta base	/api/v1/auth, /api/v1/usuarios
Swagger	http://localhost:8081/swagger-ui.html
Controller principal	AuthController, UsuarioController
Service principal	UsuarioService, CustomUserDetailsService
Repository principal	UsuarioRepository
Modelo/entidad principal	Usuario
DTOs usados	RegisterRequest
Requiere token	Parcial. /auth/login y /auth/register son publicos; usuarios requiere JWT.
Dependencias	No consume otros microservicios.

Validaciones y reglas

- Usuario: @NotBlank en nombre, password y rol.
- RegisterRequest: @NotBlank en nombre y password.
- Service: register fuerza rol USER y encripta password con BCrypt; save/update normalizan rol a mayusculas.
- No implementado actualmente: email, longitud minima de password, validacion formal de roles fuera de save/update si se usa register.
- Login autentica por nombre/password y devuelve JWT con nombre y rol.
- Swagger/OpenAPI es publico.
- Authorize en Swagger debe usarse como Bearer token: pegar el JWT devuelto por /api/v1/auth/login.

Body JSON base

```
{"nombre":"admin","password":"admin123","rol":"ADMIN"}
```

Endpoints Swagger - ms-auth-usuarios-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
POS T	/api/v1/auth/register	register	No	Publico	-	-	{"nombre":"juan","password":"123456"}	201 con message, idUsuario, nombre, rol USER	400 validacion; 500 error interno
POS T	/api/v1/auth/login	login	No	Publico	-	-	{"nombre":"juan","password":"123456"}	200 con token, nombre y rol	401 credenciales invalidas; 500
GET	/api/v1/auth/validate	validateToken	Si	USER/ADMIN autenticado	-	-	-	200 {valid:true}	401 sin token
GET	/api/v1/usuarios	findAll	Si	ADMIN	-	-	-	200 lista usuarios	401, 403
GET	/api/v1/usuarios/{id}	findById	Si	ADMIN en controller; SecurityConfig tambien permite USER/ADMIN para GET exacto	id	-	-	200 usuario	401, 403, 404
GET	/api/v1/usuarios/rol/{rol}	findByRol	Si	ADMIN	rol	-	-	200 lista	401, 403
POS T	/api/v1/usuarios	save	Si	ADMIN	-	-	{"nombre":"operador","password":"123456","rol":"USER"}	201 usuario	400, 401, 403
PUT	/api/v1/usuarios/{id}	update	Si	ADMIN	id	-	{"nombre":"operador2","password":"123456","rol":"ADMIN"}	200 usuario	400, 401, 403, 404
DEL ETE	/api/v1/usuarios/{id}	delete	Si	ADMIN	id	-	-	204	401, 403, 404

ms-cliente-service

Campo	Valor
Puerto	8082
Ruta base	/api/v1/clientes
Swagger	http://localhost:8082/swagger-ui.html
Controller principal	ClienteController
Service principal	ClienteService
Repository principal	ClienteRepository
Modelo/entidad principal	Cliente
DTOs usados	No tiene DTOs propios
Requiere token	Si. Todos los endpoints de negocio requieren JWT y @PreAuthorize ADMIN.
Dependencias	No consume otros microservicios.

Validaciones y reglas

- Cliente: @NotBlank en rutDocumento, telefono, direccion, rolCliente y tipoCliente.
- rutDocumento es unique a nivel de columna.
- Service: no hay validacion manual de formato RUT, telefono, rolCliente o tipoCliente.
- Buscar cliente inexistente por id o rut devuelve 404 mediante NoSuchElementException.
- Filtros por rolCliente y tipoCliente devuelven lista, aunque este vacia.

Body JSON base

```
{"rutDocumento":"11111111-1", "telefono":"+56911111111", "direccion":"Av. Siempre Viva 123", "rolCliente":"TITULAR", "tipoCliente":"NACIONAL"}
```

Endpoints Swagger - ms-cliente-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/clientes	findAll	Si	ADMIN	-	rolCliente?, tipoCliente?	-	200 lista	401, 403
GET	/api/v1/clientes/{id}	findById	Si	ADMIN	id	-	-	200 cliente	401, 403, 404
GET	/api/v1/clientes/rut/{rutDocu mento}	findByRutDocu mento	Si	ADMIN	rutDocument o	-	-	200 cliente	401, 403, 404
POS T	/api/v1/clientes	guardarCliente	Si	ADMIN	-	-	{"rutDocumento":"22222222-2", "telefono":"+5692222222", "direccion":"Call e 2", "rolCliente":"TITULAR", "tipoClien te":"NACIONAL"}	201 cliente	400, 401, 403
PUT	/api/v1/clientes/{id}	actualizarClient e	Si	ADMIN	id	-	{"rutDocumento":"22222222-2", "telefono":"+5693333333", "direccion":"Nue va direccion", "rolCliente":"TITULAR", " tipoCliente":"EXTRANJERO"}	200 cliente	400, 401, 403, 404
DEL ETE	/api/v1/clientes/{id}	eliminarCliente	Si	ADMIN	id	-	-	204	401, 403, 404

ms-hotel-service

Campo	Valor
Puerto	8083
Ruta base	/api/v1/hoteles
Swagger	http://localhost:8083/swagger-ui.html
Controller principal	HotelController
Service principal	HotelService
Repository principal	HotelRepository
Modelo/entidad principal	Hotel
DTOs usados	No tiene DTOs propios
Requiere token	GET publico; POST, PUT y DELETE requieren JWT ADMIN.
Dependencias	No consume otros microservicios.

Validaciones y reglas

- Hotel: @NotBlank en nombre, direccion, ciudad, pais, categoria y descripcion.
- Columnas con longitudes: nombre 50, direccion 100, ciudad 50, pais 50, categoria 30, descripcion 255.
- Service: no valida rango de estrellas ni duplicidad de hotel.
- Consultas GET son publicas por SecurityConfig.
- Filtros por categoria, ciudad y pais usan RequestParam especifico en el mismo path base.

Body JSON base

```
{"nombre":"Hotel Transilvania","direccion":"Camino del Bosque 1","ciudad":"Santiago","pais":"Chile","categoria":"5 estrellas","descripcion":"Hotel tematico"}
```

Endpoints Swagger - ms-hotel-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/hoteles	findAll	No	Publico	-	-	-	200 lista	404 si service busca id inexistente no aplica aqui
GET	/api/v1/hoteles/{id}	findById	No	Publico	id	-	-	200 hotel	404
GET	/api/v1/hoteles?categoria=5 estrellas	findByCategori a	No	Publico	-	categoria	-	200 lista	500 si param ambiguo con otros
GET	/api/v1/hoteles?ciudad=Santia go	findByCiudad	No	Publico	-	ciudad	-	200 lista	500 si param ambiguo con otros
GET	/api/v1/hoteles?pais=Chile	findByPais	No	Publico	-	pais	-	200 lista	500 si param ambiguo con otros
POS T	/api/v1/hoteles	guardarHotel	Si	ADMIN	-	-	{"nombre":"Hotel Centro","direccion":"Av. Centro 100","ciudad":"Santiago","pais":"Chile","cat egoria":"4 estrellas","descripcion":"Hotel urbano"}	201 hotel	400, 401, 403
PUT	/api/v1/hoteles/{id}	actualizarHotel	Si	ADMIN	id	-	{"nombre":"Hotel Centro Plus","direccion":"Av. Centro 100","ci udad":"Santiago","pais":"Chile","categ oria":"5 estrellas","descripcion":"Actualizado"}	200 hotel	400, 401, 403, 404
DEL ETE	/api/v1/hoteles/{id}	eliminarHotel	Si	ADMIN	id	-	-	204	401, 403, 404

ms-habitacion-service

Campo	Valor
Puerto	8084
Ruta base	/api/v1/habitaciones
Swagger	http://localhost:8084/swagger-ui.html
Controller principal	HabitacionController
Service principal	HabitacionService
Repository principal	HabitacionRepository
Modelo/entidad principal	Habitacion
DTOs usados	HotelDTO
Requiere token	GET protegido por SecurityConfig general aunque el controller no tenga @PreAuthorize; POST, PUT y DELETE requieren ADMIN.
Dependencias	WebClient a ms-hotel-service para validar idHotel.

Validaciones y reglas

- Habitacion: @NotBlank en numeroHabitacion, tipoHabitacion, estadoHabitacion; @NotNull en precioBase, capacidad e idHotel.
- numeroHabitacion es unique a nivel de columna.
- Service: estadoHabitacion debe ser DISPONIBLE, OCUPADA o MANTENIMIENTO.
- Service: valida que idHotel exista via HotelClient.
- No implementado actualmente: @Positive para precioBase, @Min(1) para capacidad.
- No se puede crear habitacion sin hotel valido.
- No se valida que la habitacion pertenezca a hotel en este servicio mas alla del idHotel enviado; esa relacion se verifica en reserva.

Body JSON base

```
{"numeroHabitacion": "101", "tipoHabitacion": "DOBLE", "precioBase": 75000, "capacidad": 2, "estadoHabitacion": "DISPONIBLE", "idHotel": 1}
```

Endpoints Swagger - ms-habitacion-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/habitaciones	findAll	Si	USER/ADMIN autenticado	-	-	-	200 lista	401
GET	/api/v1/habitaciones/{id}	findById	Si	USER/ADMIN autenticado	id	-	-	200 habitacion	401, 404
GET	/api/v1/habitaciones/estado/{e stadoHabitacion}	findByEstadoH abitacion	Si	USER/ADMIN autenticado	estadoHabitac ion	-	-	200 lista	400 estado invalido
GET	/api/v1/habitaciones/capacida d/{capacidad}	findByCapacida dMinima	Si	USER/ADMIN autenticado	capacidad	-	-	200 lista	401
GET	/api/v1/habitaciones/hotel/{idH otel}	findByIdHotel	Si	USER/ADMIN autenticado	idHotel	-	-	200 lista	401
POS T	/api/v1/habitaciones	guardarHabitac ion	Si	ADMIN	-	-	{"numeroHabitacion":"102","tipoHabitacion":"SUITE","precioBase":125000,"capacidad":4,"estadoHabitacion":"DISPONIBLE","idHotel":1}	201 habitacion	400, 401, 403, 503
PUT	/api/v1/habitaciones/{id}	actualizarHabitacion	Si	ADMIN	id	-	{"numeroHabitacion":"102","tipoHabitacion":"SUITE","precioBase":130000,"capacidad":4,"estadoHabitacion":"MANTENIMIENTO","idHotel":1}	200 habitacion	400, 401, 403, 404, 503
DEL ETE	/api/v1/habitaciones/{id}	eliminarHabitacion	Si	ADMIN	id	-	-	204	401, 403, 404

ms-disponibilidad-service

Campo	Valor
Puerto	8085
Ruta base	/api/v1/disponibilidades
Swagger	http://localhost:8085/swagger-ui.html
Controller principal	DisponibilidadController
Service principal	DisponibilidadService
Repository principal	DisponibilidadRepository
Modelo/entidad principal	Disponibilidad
DTOs usados	HabitacionDTO
Requiere token	Si. GET/PUT USER o ADMIN; POST/DELETE ADMIN.
Dependencias	WebClient a ms-habitacion-service para validar idHabitacion.

Validaciones y reglas

- Disponibilidad: @NotNull en idHabitacion y fecha; @NotBlank en estado.
- Service: estado debe ser DISPONIBLE, OCUPADA o MANTENIMIENTO.
- Service: valida habitacion existente via HabitacionClient.
- Service: evita duplicado por idHabitacion + fecha.
- Consulta principal para reserva: /habitacion/{idHabitacion}/fecha/{fecha}.
- La actualizacion permite USER o ADMIN, porque reserva usa este endpoint para ocupar/liberar disponibilidad.

Body JSON base

```
{"idHabitacion":1,"fecha":"2026-07-10","estado":"DISPONIBLE"}
```

Endpoints Swagger - ms-disponibilidad-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/disponibilidades	findAll	Si	USER/ADMIN	-	-	-	200 lista	401, 403
GET	/api/v1/disponibilidades/{id}	findById	Si	USER/ADMIN	id	-	-	200 disponibilidad	401, 403, 404
GET	/api/v1/disponibilidades/habitacion/{idHabitacion}/fecha/{fecha}	findByIdHabitacionAndFecha	Si	USER/ADMIN	idHabitacion, fecha	-	-	200 disponibilidad	401, 403, 404
POST	/api/v1/disponibilidades	guardar	Si	ADMIN	-	-	{"idHabitacion":1,"fecha":"2026-07-10","estado":"DISPONIBLE"}	201 disponibilidad	400, 401, 403, 409 no implementado, 503
PUT	/api/v1/disponibilidades/{id}	actualizar	Si	USER/ADMIN	id	-	{"idHabitacion":1,"fecha":"2026-07-10","estado":"OCUPADA"}	200 disponibilidad	400, 401, 403, 404, 503
DELETE	/api/v1/disponibilidades/{id}	eliminar	Si	ADMIN	id	-	-	204	401, 403, 404

ms-reserva-service

Campo	Valor
Puerto	8086
Ruta base	/api/v1/reservas
Swagger	http://localhost:8086/swagger-ui.html
Controller principal	ReservaController
Service principal	ReservaService
Repository principal	ReservaRepository
Modelo/entidad principal	Reserva
DTOs usados	ClienteDTO, UsuarioDTO, HotelDTO, HabitacionDTO, DisponibilidadDTO
Requiere token	Si. GET lista/PUT/DELETE ADMIN; GET id, POST y cambio estado USER o ADMIN.
Dependencias	WebClient a cliente, auth/usuarios, hotel, habitacion y disponibilidad.

Validaciones y reglas

- Reserva: @NotNull en ids, fechaInicio, fechaFin, cantidadPersonas; @Min(1) en cantidadPersonas; @NotBlank en estadoReserva.
- Service: estadoReserva debe ser CONFIRMADA, PENDIENTE, CANCELADA o FINALIZADA.
- Service: fechaFin debe ser posterior a fechaInicio.
- Service: valida cliente, usuario, hotel, habitacion y disponibilidad por WebClient.
- Service: valida que habitacion.idHotel coincida con idHotel de reserva.
- Service: valida habitacion DISPONIBLE y capacidad suficiente.
- Importante: al crear una reserva no se fuerza PENDIENTE; acepta el estado enviado si es valido. Recomendacion: forzar PENDIENTE en guardar.
- No se puede crear reserva si cliente, usuario, hotel o habitacion no existen.
- No se puede reservar si la habitacion no pertenece al hotel o no esta DISPONIBLE.
- No se puede confirmar si no hay disponibilidad diaria DISPONIBLE entre fechaInicio inclusive y fechaFin exclusiva.
- Cuando una reserva pasa a CONFIRMADA, marca disponibilidad como OCUPADA.
- Cuando deja de estar CONFIRMADA, se libera disponibilidad a DISPONIBLE.

Body JSON base

```
{"idCliente":1,"idUsuario":1,"idHotel":1,"idHabitacion":1,"fechaInicio":"2026-07-10","fechaFin":"2026-07-12","cantidadPersonas":2,"estadoReserva":"PENDIENTE"}
```

Endpoints Swagger - ms-reserva-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/reservas	findAll	Si	ADMIN	-	idCliente?, idUsuario?, idHotel?, idHabitacion?, estadoReserva?	-	200 lista	401, 403
GET	/api/v1/reservas/{id}	findById	Si	USER/ADMIN	id	-	-	200 reserva	401, 403, 404
POST	/api/v1/reservas	guardarReserva	Si	USER/ADMIN	-	-	{"idCliente":1,"idUsuario":1,"idHotel":1,"idHabitacion":1,"fechaInicio":"2026-07-10","fechaFin":"2026-07-12","cantidadPersonas":2,"estadoReserva":"PENDIENTE"}	201 reserva	400, 401, 403, 404 desde dependencia, 503
PUT	/api/v1/reservas/{id}	actualizarReserva	Si	ADMIN	id	-	{"idCliente":1,"idUsuario":1,"idHotel":1,"idHabitacion":1,"fechaInicio":"2026-07-11","fechaFin":"2026-07-13","cantidadPersonas":2,"estadoReserva":"CONFIRMADA"}	200 reserva	400, 401, 403, 404, 503
PUT	/api/v1/reservas/{id}/estado?estadoReserva=CONFIRMADA	cambiarEstado	Si	USER/ADMIN	id	estadoReserva	-	200 reserva; ocupa/libera disponibilidad segun estado	400, 401, 403, 404, 503
DELETE	/api/v1/reservas/{id}	eliminarReserva	Si	ADMIN	id	-	-	204; libera disponibilidad antes de eliminar	401, 403, 404, 503

ms-pago-service

Campo	Valor
Puerto	8087
Ruta base	/api/v1/pagos
Swagger	http://localhost:8087/swagger-ui.html
Controller principal	PagoController
Service principal	PagoService
Repository principal	PagoRepository
Modelo/entidad principal	Pago
DTOs usados	ReservaDTO, UsuarioDTO
Requiere token	Si. Listar y filtros por estado ADMIN; crear y consultar por id/reserva USER o ADMIN.
Dependencias	WebClient a ms-reserva-service y ms-auth-usuarios-service.

Validaciones y reglas

- Pago: @NotNull en reservaId, idUsuario, monto, fechaPago; @NotBlank en metodoPago y estadoPago.
- Service: monto debe ser mayor a 0.
- Service: estadoPago debe ser PENDIENTE, APROBADO, RECHAZADO, REEMBOLSADO o ANULADO.
- Service: valida reserva y usuario existentes por WebClient.
- No implementado actualmente: impedir dos pagos para la misma reserva.
- APROBADO cambia reserva a CONFIRMADA.
- RECHAZADO cambia reserva a CANCELADA.
- REEMBOLSADO cambia reserva a CANCELADA.
- PENDIENTE cambia reserva a PENDIENTE.
- ANULADO no cambia automaticamente el estado de la reserva.

Body JSON base

```
{"reservaId":1,"idUsuario":1,"monto":150000,"metodoPago":"TARJETA","estadoPago":"APROBADO","fechaPago":"2026-07-01T10:00:00"}
```

Endpoints Swagger - ms-pago-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/pagos	findAll	Si	ADMIN	-	-	-	200 lista	401, 403
GET	/api/v1/pagos/{id}	findById	Si	USER/ADMIN	id	-	-	200 pago	401, 403, 404
POS T	/api/v1/pagos	save	Si	USER/ADMIN	-	-	{"reservald":1,"idUser":1,"monto":150000,"metodoPago":"TARJETA","estadoPago":"APROBADO","fechaPago":"2026-07-01T10:00:00"}	201 pago; si APROBADO confirma reserva	400, 401, 403, 404, 503
PUT	/api/v1/pagos/{id}	update	Si	ADMIN	id	-	{"reservald":1,"idUser":1,"monto":150000,"metodoPago":"TRANSFERENCIA","estadoPago":"REEMBOLSADO","fechaPago":"2026-07-02T10:00:00"}	200 pago; actualiza reserva segun estado	400, 401, 403, 404, 503
DEL ETE	/api/v1/pagos/{id}	delete	Si	ADMIN	id	-	-	204	401, 403, 404
GET	/api/v1/pagos/reserva/{reservald}	findByReservald	Si	USER/ADMIN	reservald	-	-	200 lista	401, 403
GET	/api/v1/pagos/estado/{estadoPago}	findByEstadoPago	Si	ADMIN	estadoPago	-	-	200 lista	401, 403

ms-notificacion-service

Campo	Valor
Puerto	8088
Ruta base	/api/v1/notificaciones
Swagger	http://localhost:8088/swagger-ui.html
Controller principal	NotificacionController
Service principal	NotificacionService
Repository principal	NotificacionRepository
Modelo/entidad principal	Notificacion
DTOs usados	ClienteDTO, UsuarioDTO, ReservaDTO
Requiere token	Si. Listar/crear/actualizar/eliminar ADMIN; buscar por id USER o ADMIN.
Dependencias	WebClient a cliente, auth/usuarios y reserva.

Validaciones y reglas

- Notificacion: @NotNull en idCliente, idUsuario, idReserva y fechaEnvio.
- @NotBlank en tipo, mensaje y estado.
- Service: valida cliente, usuario y reserva existentes por WebClient.
- No implementado actualmente: catalogo de tipo/estado.
- No se envia notificacion real; se registra en base de datos.

Body JSON base

```
{"idCliente":1,"idUsuario":1,"idReserva":1,"tipo":"RESERVA","mensaje":"Reserva creada","estado":"PENDIENTE","fechaEnvio":"2026-07-01T10:00:00"}
```

Endpoints Swagger - ms-notificacion-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/notificaciones	findAll	Si	ADMIN	-	idCliente?, idUsuario?, idReserva?	-	200 lista	401, 403
GET	/api/v1/notificaciones/{id}	findById	Si	USER/ADMIN	id	-	-	200 notificacion	401, 403, 404
POS T	/api/v1/notificaciones	guardar	Si	ADMIN	-	-	{"idCliente":1,"idUsuario":1,"idReserva":1,"tipo":"EMAIL","mensaje":"Pago aprobado","estado":"ENVIADA","fechaEnvio":"2026-07-01T10:00:00"}	201 notificacion	400, 401, 403, 404, 503
PUT	/api/v1/notificaciones/{id}	actualizar	Si	ADMIN	id	-	{"idCliente":1,"idUsuario":1,"idReserva":1,"tipo":"EMAIL","mensaje":"Actualizada","estado":"LEIDA","fechaEnvio":"2026-07-01T10:00:00"}	200 notificacion	400, 401, 403, 404, 503
DEL ETE	/api/v1/notificaciones/{id}	eliminar	Si	ADMIN	id	-	-	204	401, 403, 404

ms-resena-service

Campo	Valor
Puerto	8089
Ruta base	/api/v1/resenas
Swagger	http://localhost:8089/swagger-ui.html
Controller principal	ResenaController
Service principal	ResenaService
Repository principal	ResenaRepository
Modelo/entidad principal	Resena
DTOs usados	ClienteDTO, HotelDTO, HabitacionDTO
Requiere token	Si. Casi todo USER o ADMIN; filtro por estado ADMIN.
Dependencias	WebClient a cliente, hotel y habitacion. No valida reserva aunque idReserva es obligatorio.

Validaciones y reglas

- Resena: @NotNull en idCliente, idHotel, idHabitacion, idReserva y calificacion.
- @Min(1) y @Max(5) en calificacion; @NotBlank en comentario y estadoResena.
- @PrePersist asigna fechaComentario automaticamente.
- Service: valida cliente, hotel y habitacion existentes por WebClient.
- No implementado actualmente: validar reserva existente, que reserva pertenezca al cliente/hotel/habitacion, catalogo de estados.
- Servicio real existente. No existe ms-comentario-service en el proyecto.

Body JSON base

```
{"idCliente":1,"idHotel":1,"idHabitacion":1,"idReserva":1,"calificacion":5,"comentario":"Excelente estadia","estadoResena":"PUBLICADA"}
```

Endpoints Swagger - ms-resena-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/resenas	listarResenas	Si	USER/ADMIN	-	-	-	200 lista	401, 403
GET	/api/v1/resenas/{id}	buscarPorId	Si	USER/ADMIN	id	-	-	200 resena	401, 403, 404
POST	/api/v1/resenas	guardarResena	Si	USER/ADMIN	-	-	{ "idCliente":1, "idHotel":1, "idHabitacion":1, "idReserva":1, "calificacion":5, "comentario": "Excelente", "estadoResena": "PUBLICADA" }	200 resena creada	400, 401, 403, 404, 503
PUT	/api/v1/resenas/{id}	actualizarResena	Si	USER/ADMIN	id	-	{ "idCliente":1, "idHotel":1, "idHabitacion":1, "idReserva":1, "calificacion":4, "comentario": "Muy bueno", "estadoResena": "PUBLICADA" }	200 resena	400, 401, 403, 404, 503
DELETE	/api/v1/resenas/{id}	eliminarResena	Si	USER/ADMIN	id	-	-	200 texto	401, 403, 404
GET	/api/v1/resenas/cliente/{idCliente}	buscarPorCliente	Si	USER/ADMIN	idCliente	-	-	200 lista	401, 403
GET	/api/v1/resenas/hotel/{idHotel}	buscarPorHotel	Si	USER/ADMIN	idHotel	-	-	200 lista	401, 403
GET	/api/v1/resenas/habitacion/{idHabitacion}	buscarPorHabitacion	Si	USER/ADMIN	idHabitacion	-	-	200 lista	401, 403
GET	/api/v1/resenas/reserva/{idReserva}	buscarPorReserva	Si	USER/ADMIN	idReserva	-	-	200 lista	401, 403
GET	/api/v1/resenas/calificacion/{calificacion}	buscarPorCalificacion	Si	USER/ADMIN	calificacion	-	-	200 lista	401, 403
GET	/api/v1/resenas/estado/{estadoResena}	buscarPorEstado	Si	ADMIN	estadoResena	-	-	200 lista	401, 403

ms-servicio-adicional-service

Campo	Valor
Puerto	8090
Ruta base	/api/v1/servicios-adicionales
Swagger	http://localhost:8090/swagger-ui.html
Controller principal	ServicioAdicionalController
Service principal	ServicioAdicionalService
Repository principal	ServicioAdicionalRepository
Modelo/entidad principal	ServicioAdicionalModel
DTOs usados	HotelDTO, ReservaDTO
Requiere token	Si. GET USER o ADMIN; crear/actualizar/eliminar ADMIN.
Dependencias	WebClient a hotel y reserva.

Validaciones y reglas

- ServicioAdicionalModel: @NotNull en idHotel y precio; @NotBlank en nombre, descripcion y estado.
- idReserva es opcional.
- Service: valida hotel existente y, si idReserva no es null, valida reserva existente.
- No implementado actualmente: precio positivo, estados permitidos, relacion reserva-hotel.
- Puede existir servicio asociado solo al hotel o tambien a una reserva.

Body JSON base

```
{"idHotel":1,"idReserva":1,"nombre":"Spa","descripcion":"Acceso a spa","precio":25000,"estado":"ACTIVO"}
```

Endpoints Swagger - ms-servicio-adicional-service

HTT P	URL	Metodo	JWT	Roles	PathVariable	RequestParam	Body JSON	Exito	Errores
GET	/api/v1/servicios-adicionales	findAll	Si	USER/ADMIN	-	idHotel?, idReserva?, estado?, nombre?	-	200 lista	401, 403
GET	/api/v1/servicios-adicionales/{id}	findById	Si	USER/ADMIN	id	-	-	200 servicio	401, 403, 404
POST	/api/v1/servicios-adicionales	guardarServicio	Si	ADMIN	-	-	{"idHotel":1,"idReserva":1,"nombre":"Desayuno","descripcion":"Desayuno buffet","precio":12000,"estado":"ACTIVO"}	201 servicio	400, 401, 403, 404, 503
PUT	/api/v1/servicios-adicionales/{id}	actualizarServicio	Si	ADMIN	id	-	{"idHotel":1,"idReserva":1,"nombre":"Desayuno premium","descripcion":"Buffet premium","precio":15000,"estado":"ACTIVO"}	200 servicio	400, 401, 403, 404, 503
DELETE	/api/v1/servicios-adicionales/{id}	eliminarServicio	Si	ADMIN	id	-	-	204	401, 403, 404

Modelos Y Entidades Por Microservicio

- Los modelos revisados son entidades JPA reales ubicadas en src/main/java de cada microservicio.
- No se encontraron relaciones JPA como @ManyToOne o @OneToMany entre modelos. Las relaciones entre microservicios se expresan con campos id y se validan, cuando aplica, mediante WebClient en los services.
- api-gateway y eureka-server no tienen entidad de negocio JPA en el código revisado.
- ms-comentario-service no existe en el proyecto actual. El servicio equivalente encontrado es ms-resena-service con la entidad Resena.
- El campo _links aparece en respuestas por HATEOAS, pero no debe enviarse al crear objetos desde Swagger.

ms-auth-usuarios-service - Usuario

Campo	Detalle
Modelo o entidad	Usuario
Tabla BD	usuario
Obligatorios al crear	nombre, password y rol si se usa /api/v1/usuarios. En /api/v1/auth/register solo nombre y password; rol lo asigna el sistema como USER.
No enviar en JSON	idUsuario, _links. En register no enviar rol porque no existe en RegisterRequest.

Tabla de atributos

Atributos - Usuario

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
idUsuario	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idUsuario)	No aplica	No	Identificador unico del usuario generado por la base de datos.	1	abc
nombre	String	No	No	Si	@NotBlank, @Column(nullable=false)	No aplica	No	Nombre de usuario usado para login.	admin	
password	String	No	No	Si	@NotBlank, @Column(nullable=false)	No aplica	No	Contraseña del usuario; el service la guarda encriptada con BCrypt.	admin123	
rol	String	No	No	Si en Usuario	@NotBlank, @Column(nullable=false)	No aplica	Rol	Rol de autorizacion usado por Spring Security.	ADMIN	INVITADO

ms-cliente-service - Cliente

Campo	Detalle
Modelo o entidad	Cliente
Tabla BD	cliente
Obligatorios al crear	rutDocumento, telefono, direccion, rolCliente, tipoCliente.
No enviar en JSON	id, _links.

Tabla de atributos

Atributos - Cliente

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
id	Long	ID si	Autoincre mental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idCliente)	No aplica	No	Identificador del cliente.	1	abc
rutDocumento	String	No	No	Si	@NotBlank, @Column(nullable=false, unique=true)	No aplica	Tipo/documento	Documento unico del cliente.	11111111-1	
telefono	String	No	No	Si	@NotBlank, @Column(nullable=false)	No aplica	No	Telefono de contacto.	+56911111111	
direccion	String	No	No	Si	@NotBlank, @Column(nullable=false)	No aplica	No	Direccion del cliente.	Av. Siempre Viva 123	
rolCliente	String	No	No	Si	@NotBlank, @Column(nullable=false)	No aplica	Rol/tipo	Rol funcional del cliente.	TITULAR	
tipoCliente	String	No	No	Si	@NotBlank, @Column(nullable=false)	No aplica	Tipo	Clasificacion del cliente.	NACIONAL	

ms-hotel-service - Hotel

Campo	Detalle
Modelo o entidad	Hotel
Tabla BD	hotel
Obligatorios al crear	nombre, direccion, ciudad, pais, categoria, descripcion.
No enviar en JSON	id, _links.

Tabla de atributos

Atributos - Hotel

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
id	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idHotel)	No aplica	No	Identificador del hotel.	1	abc
nombre	String	No	No	Si	@NotBlank, @Column(nullable=false,length=50)	No aplica	No	Nombre comercial del hotel.	Hotel Transilvania	
direccion	String	No	No	Si	@NotBlank, @Column(nullable=false,length=100)	No aplica	No	Direccion fisica.	Camino del Bosque 1	
ciudad	String	No	No	Si	@NotBlank, @Column(nullable=false,length=50)	No aplica	No	Ciudad del hotel.	Santiago	
pais	String	No	No	Si	@NotBlank, @Column(nullable=false,length=50)	No aplica	No	Pais del hotel.	Chile	
categoria	String	No	No	Si	@NotBlank, @Column(nullable=false,length=30)	No aplica	Tipo	Categoria comercial; no hay enum ni rango implementado.	5 estrellas	
descripcion	String	No	No	Si	@NotBlank, @Column(nullable=false,length=255)	No aplica	No	Descripcion breve del hotel.	Hotel tematico familiar	

ms-habitacion-service - Habitacion

Campo	Detalle
Modelo o entidad	Habitacion
Tabla BD	habitacion
Obligatorios al crear	numeroHabitacion, tipoHabitacion, precioBase, capacidad, estadoHabitacion, idHotel.
No enviar en JSON	idHabitacion, _links.

Tabla de atributos

Atributos - Habitación

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
idHabitacion	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idHabitacion)	No aplica	No	Identificador de la habitacion.	1	abc
numeroHabitacion	String	No	No	Si	@NotBlank, @Column(nullable=false, unique=true)	No aplica	No	Numero o codigo visible de habitacion.	101	
tipoHabitacion	String	No	No	Si	@NotBlank	No aplica	Tipo	Tipo comercial de habitacion.	DOBLE	
precioBase	Double	No	No	Si	@NotNull	No aplica	No	Precio base; no tiene @Positive en el modelo actual.	75000	-100
capacidad	Integer	No	No	Si	@NotNull	No aplica	No	Cantidad maxima de personas; no tiene @Min en el modelo actual.	2	0
estadoHabitacion	String	No	No	Si	@NotBlank; service valida DISPONIBLE, OCUPADA, MANTENIMIENTO	No aplica	Estado	Estado operativo de la habitacion.	DISPONIBLE	BLOQUEADA
idHotel	Long	No	No	Si	@NotNull	ms-hotel-service	No	Hotel al que pertenece la habitacion.	1	999999

ms-disponibilidad-service - Disponibilidad

Campo	Detalle
Modelo o entidad	Disponibilidad
Tabla BD	disponibilidad
Obligatorios al crear	idHabitacion, fecha, estado.
No enviar en JSON	id, _links.

Tabla de atributos

Atributos - Disponibilidad

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
id	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idDisponibilidad)	No aplica	No	Identificador de disponibilidad diaria.	1	abc
idHabitacion	Long	No	No	Si	@NotNull	ms-habitacion-servicio	No	Habitacion cuya disponibilidad se controla.	1	999999
fecha	LocalDate	No	No	Si	@NotNull	No aplica	No	Dia disponible u ocupado.	2026-07-10	fecha-mal
estado	String	No	No	Si	@NotBlank; service valida DISPONIBLE, OCUPADA, MANTENIMIENTO	No aplica	Estado	Estado de esa habitacion para esa fecha.	DISPONIBLE	RESERVADA

ms-reserva-service - Reserva

Campo	Detalle
Modelo o entidad	Reserva
Tabla BD	reserva
Obligatorios al crear	idCliente, idUsuario, idHotel, idHabitacion, fechaInicio, fechaFin, cantidadPersonas, estadoReserva. fechaCreacion esta marcada non-null en columna, pero el service la asigna automaticamente.
No enviar en JSON	id, fechaCreacion, _links. Recomendacion: crear con estadoReserva PENDIENTE aunque el codigo acepta otros estados validos.

Tabla de atributos

Atributos - Reserva

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
id	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idReserva)	No aplica	No	Identificador de la reserva.	1	abc
idCliente	Long	No	No	Si	@NotNull	ms-cliente-service	No	Cliente asociado.	1	999999
idUsuario	Long	No	No	Si	@NotNull	ms-auth-usuarios-service	No	Usuario que registra o gestiona la reserva.	1	999999
idHotel	Long	No	No	Si	@NotNull	ms-hotel-service	No	Hotel de la reserva.	1	999999
idHabitacion	Long	No	No	Si	@NotNull	ms-habitacion-service	No	Habitacion reservada.	1	999999
fechaInicio	LocalDate	No	No	Si	@NotNull; service exige fechaFin posterior	No aplica	No	Primer dia de estadia.	2026-07-10	2026-07-12 con fin 2026-07-10
fechaFin	LocalDate	No	No	Si	@NotNull; debe ser posterior a fechaInicio	No aplica	No	Dia de salida; se usa como limite exclusivo.	2026-07-12	2026-07-10
cantidadPersonas	Integer	No	No	Si	@NotNull, @Min(1); service valida capacidad	Habitacion.capacidad	No	Cantidad de huespedes.	2	0
estadoReserva	String	No	No	Si	@NotBlank; service valida CONFIRMADA, PENDIENTE, CANCELADA, FINALIZADA	Disponibilidad si CONFIRMADA	Estado	Estado del ciclo de vida de la reserva.	PENDIENTE	NUEVA
fechaCreacion	LocalDateTime	No	No	Sistema	@Column(nullable=false); service setea LocalDateTime.now()	No aplica	No	Fecha/hora en que se crea la reserva.	2026-07-01T10:00:00	null si no lo setea el service

ms-pago-service - Pago

Campo	Detalle
Modelo o entidad	Pago
Tabla BD	pago
Obligatorios al crear	reservald, idUsuario, monto, metodoPago, estadoPago, fechaPago. Si fechaPago llega null, el service intenta asignar ahora antes de validar.
No enviar en JSON	idPago, _links. fechaPago puede enviarse o dejarse para que el service la complete, aunque el modelo declara @NotNull.

Tabla de atributos

Atributos - Pago

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
idPago	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idPago)	No aplica	No	Identificador del pago.	1	abc
reservald	Long	No	No	Si	@NotNull	ms-reserva-service	No	Reserva que se paga.	1	999999
idUsuario	Long	No	No	Si	@NotNull	ms-auth-usuarios-service	No	Usuario que registra el pago.	1	999999
monto	BigDecimal	No	No	Si	@NotNull; service exige > 0	No aplica	No	Monto del pago.	150000	0
metodoPago	String	No	No	Si	@NotBlank	No aplica	Tipo	Medio usado para pagar.	TARJETA	
estadoPago	String	No	No	Si	@NotBlank; service valida PENDIENTE, APROBADO, RECHAZADO, REEMBOLSADO, ANULADO	Reserva cambia de estado	Estado	Resultado/ciclo del pago.	APROBADO	PAGADO
fechaPago	LocalDateTime	No	No	Si o sistema	@NotNull; service asigna now si null	No aplica	No	Fecha/hora del pago.	2026-07-01T10:00:00	fecha-mal

ms-notificacion-service - Notificacion

Campo	Detalle
Modelo o entidad	Notificacion
Tabla BD	notificacion
Obligatorios al crear	idCliente, idUsuario, idReserva, tipo, mensaje, estado, fechaEnvio.
No enviar en JSON	id, _links.

Tabla de atributos

Atributos - Notificacion

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
id	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idNotificacion)	No aplica	No	Identificador de notificacion.	1	abc
idCliente	Long	No	No	Si	@NotNull	ms-cliente-service	No	Cliente destinatario o asociado.	1	999999
idUsuario	Long	No	No	Si	@NotNull	ms-auth-usuarios-service	No	Usuario asociado.	1	999999
idReserva	Long	No	No	Si	@NotNull	ms-reserva-service	No	Reserva relacionada.	1	999999
tipo	String	No	No	Si	@NotBlank	No aplica	Tipo	Canal o tipo de notificacion; no hay enum implementado.	EMAIL	
mensaje	String	No	No	Si	@NotBlank	No aplica	No	Texto de la notificacion.	Pago aprobado	
estado	String	No	No	Si	@NotBlank	No aplica	Estado	Estado textual; no hay catalogo implementado.	ENVIADA	
fechaEnvio	LocalDateTime	No	No	Si	@NotNull	No aplica	No	Fecha/hora de envio registrada.	2026-07-01T10:00:00	fecha-mal

ms-resena-service - Resena

Campo	Detalle
Modelo o entidad	Resena
Tabla BD	resena
Obligatorios al crear	idCliente, idHotel, idHabitacion, idReserva, calificacion, comentario, estadoResena.
No enviar en JSON	id, fechaComentario, _links. fechaComentario se asigna con @PrePersist.

Tabla de atributos

Atributos - Resena

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
id	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=id_resena)	No aplica	No	Identificador de resena.	1	abc
idCliente	Long	No	No	Si	@NotNull	ms-cliente-service	No	Cliente autor.	1	999999
idHotel	Long	No	No	Si	@NotNull	ms-hotel-service	No	Hotel reseñado.	1	999999
idHabitacion	Long	No	No	Si	@NotNull	ms-habitacion-service	No	Habitacion reseñada.	1	999999
idReserva	Long	No	No	Si	@NotNull	No validado por WebClient actual	No	Reserva relacionada; el service no valida que exista.	1	999999
calificacion	Integer	No	No	Si	@NotNull, @Min(1), @Max(5)	No aplica	No	Puntaje de 1 a 5.	5	6
comentario	String	No	No	Si	@NotBlank, @Column(length=500)	No aplica	No	Comentario escrito.	Excelente	
fechaComentario	LocalDate	No	No	Sistema	@PrePersist asigna LocalDate.now()	No aplica	No	Fecha automatica de creacion.	2026-07-01	No enviar
estadoResena	String	No	No	Si	@NotBlank	No aplica	Estado	Estado textual de la resena; no hay enum.	PUBLICADA	

ms-servicio-adicional-service - ServicioAdicionalModel

Campo	Detalle
Modelo o entidad	ServicioAdicionalModel
Tabla BD	servicio_adicional
Obligatorios al crear	idHotel, nombre, descripcion, precio, estado. idReserva es opcional.
No enviar en JSON	id, _links.

Tabla de atributos

Atributos - ServicioAdicionalModel

Atributo	Tipo	ID	Auto	Obligatorio	Validacion / columna	Relacion	Estado/rol/tipo	Explicacion	Ejemplo valido	Ejemplo invalido
id	Long	ID si	Autoincremental si	No enviar	@Id, @GeneratedValue(IDENTITY), @Column(name=idServicio)	No aplica	No	Identificador del servicio adicional.	1	abc
idHotel	Long	No	No	Si	@NotNull, @Column(nullable=false)	ms-hotel-service	No	Hotel que ofrece el servicio.	1	999999
idReserva	Long	No	No	Opcional	@Column(nullable=true)	ms-reserva-service si se envia	No	Reserva asociada si el servicio ya fue contratado.	1	null permitido
nombre	String	No	No	Si	@NotBlank, @Column(length=100)	No aplica	No	Nombre del servicio.	Spa	
descripcion	String	No	No	Si	@NotBlank, @Column(length=200)	No aplica	No	Detalle del servicio.	Acceso a spa	
precio	BigDecimal	No	No	Si	@NotNull	No aplica	No	Precio del servicio; no tiene @Positive.	25000	-1
estado	String	No	No	Si	@NotBlank, @Column(length=20)	No aplica	Estado	Estado textual; no hay enum implementado.	ACTIVO	

Datos Necesarios Para Poblar Cada Microservicio

- Los ejemplos siguientes preparan datos base para crear 5 objetos por microservicio desde Swagger.
- Respeta el orden porque varios servicios validan IDs en otros microservicios mediante WebClient.
- Ajusta los IDs si tu base de datos ya tiene registros previos o si el autoincremental no parte en 1.

Orden global de poblamiento

- 1. Crear o sembrar usuarios, incluyendo al menos un ADMIN.
- 2. Crear hoteles.
- 3. Crear clientes.
- 4. Crear habitaciones asociadas a hoteles existentes.
- 5. Crear disponibilidades asociadas a habitaciones existentes.
- 6. Crear reservas usando cliente, usuario, hotel, habitacion y disponibilidad validos.
- 7. Crear pagos asociados a reservas existentes.
- 8. Crear notificaciones, reseñas y servicios adicionales cuando ya existan sus dependencias.

ms-auth-usuarios-service

Campo	Detalle
Estructura JSON esperada	<code>{"nombre":"admin","password":"admin123","rol":"ADMIN"}</code>
Reglas para poblar	- Para login se necesita que el usuario exista. - register crea siempre rol USER. - Para crear usuarios ADMIN desde Swagger se requiere token ADMIN en /api/v1/usuarios.
Dependencias previas	No depende de otros microservicios.
Orden correcto	Primero crear o sembrar ADMIN; luego crear usuarios USER si se necesita.
5 objetos recomendados	<pre> {"nombre":"admin","password":"admin123","rol":"ADMIN"} {"nombre":"repcion","password":"recep123","rol":"USER"} {"nombre":"ventas","password":"ventas123","rol":"USER"} {"nombre":"supervisor","password":"super123","rol":"ADMIN"} {"nombre":"clienteapp","password":"cliente123","rol":"USER"} </pre>

ms-cliente-service

Campo	Detalle
Estructura JSON esperada	<code>{"rutDocumento":"11111111-1","telefono":"+56911111111","direccion":"Av. Siempre Viva 123","rolCliente":"TITULAR","tipoCliente":"NACIONAL"}</code>
Reglas para poblar	- rutDocumento debe ser unico. - El modelo no valida formato RUT ni telefono; usar datos coherentes para pruebas.
Dependencias previas	No depende de otros microservicios.
Orden correcto	Crear despues de obtener token ADMIN y antes de reservas, reseñas y notificaciones.
5 objetos recomendados	<pre> {"rutDocumento":"11111111-1","telefono":"+56911111111","direccion":"Av. Siempre Viva 123","rolCliente":"TITULAR","tipoCliente":"NACIONAL"} {"rutDocumento":"22222222-2","telefono":"+56922222222","direccion":"Calle Luna 45","rolCliente":"TITULAR","tipoCliente":"NACIONAL"} {"rutDocumento":"33333333-3","telefono":"+56933333333","direccion":"Pasaje Sol 10","rolCliente":"ACOMPANANTE","tipoCliente":"EXTRANJERO"} {"rutDocumento":"44444444-4","telefono":"+56944444444","direccion":"Ruta 5 Km 20","rolCliente":"EMPRESA","tipoCliente":"NACIONAL"} {"rutDocumento":"55555555-5","telefono":"+56955555555","direccion":"Los Pinos 777","rolCliente":"TITULAR","tipoCliente":"EXTRANJERO"} </pre>

ms-hotel-service

Campo	Detalle
Estructura JSON esperada	<code>{"nombre":"Hotel Transilvania","direccion":"Camino del Bosque 1","ciudad":"Santiago","pais":"Chile","categoria":"5 estrellas","descripcion":"Hotel tematico"}</code>

Campo	Detalle
Reglas para poblar	- GET es publico, pero crear/editar/eliminar requiere ADMIN. - No hay validacion numerica de estrellas; categoria es texto.
Dependencias previas	No depende de otros microservicios.
Orden correcto	Crear antes de habitaciones, servicios adicionales, reservas, reseñas.
5 objetos recomendados	<pre>{ "nombre": "Hotel Transilvania", "direccion": "Camino del Bosque 1", "ciudad": "Santiago", "pais": "Chile", "categoria": "5 estrellas", "descripcion": "Hotel tematico" } {"nombre": "Hotel Castillo", "direccion": "Calle Niebla 22", "ciudad": "Valparaiso", "pais": "Chile", "categoria": "4 estrellas", "descripcion": "Hotel patrimonial"} {"nombre": "Hotel Lago", "direccion": "Costanera 100", "ciudad": "Puerto Varas", "pais": "Chile", "categoria": "4 estrellas", "descripcion": "Vista al lago"} {"nombre": "Hotel Desierto", "direccion": "Av. Oasis 8", "ciudad": "San Pedro", "pais": "Chile", "categoria": "3 estrellas", "descripcion": "Hotel de aventura"} {"nombre": "Hotel Austral", "direccion": "Ruta Austral 50", "ciudad": "Punta Arenas", "pais": "Chile", "categoria": "5 estrellas", "descripcion": "Hotel austral"}</pre>

ms-habitacion-service

Campo	Detalle
Estructura JSON esperada	<pre>{ "numeroHabitacion": "101", "tipoHabitacion": "DOBLE", "precioBase": 75000, "capacidad": 2, "estadoHabitacion": "DISPONIBLE", "idHotel": 1 }</pre>
Reglas para poblar	- Debe existir hotel con idHotel. - estadoHabitacion debe ser DISPONIBLE, OCUPADA o MANTENIMIENTO. - numeroHabitacion debe ser unico.
Dependencias previas	Requiere hotel existente.
Orden correcto	Crear hoteles antes; luego habitaciones.
5 objetos recomendados	<pre>{ "numeroHabitacion": "101", "tipoHabitacion": "SIMPLE", "precioBase": 50000, "capacidad": 1, "estadoHabitacion": "DISPONIBLE", "idHotel": 1 } {"numeroHabitacion": "102", "tipoHabitacion": "DOBLE", "precioBase": 75000, "capacidad": 2, "estadoHabitacion": "DISPONIBLE", "idHotel": 1} {"numeroHabitacion": "201", "tipoHabitacion": "SUITE", "precioBase": 125000, "capacidad": 4, "estadoHabitacion": "DISPONIBLE", "idHotel": 1} {"numeroHabitacion": "301", "tipoHabitacion": "FAMILIAR", "precioBase": 100000, "capacidad": 5, "estadoHabitacion": "MANTENIMIENTO", "idHotel": 2} {"numeroHabitacion": "401", "tipoHabitacion": "DOBLE", "precioBase": 82000, "capacidad": 2, "estadoHabitacion": "OCUPADA", "idHotel": 2}</pre>

ms-disponibilidad-service

Campo	Detalle
Estructura JSON esperada	<pre>{ "idHabitacion": 1, "fecha": "2026-07-10", "estado": "DISPONIBLE" }</pre>
Reglas para poblar	- Debe existir habitacion. - No duplicar misma idHabitacion y fecha. - Usar DISPONIBLE para fechas que se quieren reservar.
Dependencias previas	Requiere habitacion existente.
Orden correcto	Crear habitaciones antes; luego crear disponibilidad por fecha.
5 objetos recomendados	<pre>{ "idHabitacion": 1, "fecha": "2026-07-10", "estado": "DISPONIBLE" } {"idHabitacion": 1, "fecha": "2026-07-11", "estado": "DISPONIBLE"} {"idHabitacion": 2, "fecha": "2026-07-10", "estado": "DISPONIBLE"} {"idHabitacion": 2, "fecha": "2026-07-11", "estado": "OCUPADA"} {"idHabitacion": 3, "fecha": "2026-07-12", "estado": "MANTENIMIENTO"}</pre>

ms-reserva-service

Campo	Detalle
Estructura JSON esperada	<pre>{ "idCliente": 1, "idUsuario": 1, "idHotel": 1, "idHabitacion": 1, "fechaInicio": "2026-07-10", "fechaFin": "2026-07-12", "cantidadPersonas": 2, "estadoReserva": "PENDIENTE" }</pre>
Reglas para poblar	- Deben existir cliente, usuario, hotel y habitacion. - La habitacion debe pertenecer al hotel. - La habitacion debe estar DISPONIBLE. - Si se confirma, debe haber disponibilidad DISPONIBLE para cada día. - fechaFin debe ser posterior a fechaInicio.

Campo	Detalle
Dependencias previas	Cliente, usuario, hotel, habitacion y disponibilidad.
Orden correcto	Crear usuario/admin, hotel, cliente, habitacion y disponibilidad antes de reserva.
5 objetos recomendados	<pre>{ "idCliente":1,"idUsuario":1,"idHotel":1,"idHabitacion":1,"fechaInicio":"2026-07-10","fechaFin":"2026-07-12","cantidadPersonas":1,"estadoReserva":"PENDIENTE"} {"idCliente":2,"idUsuario":1,"idHotel":1,"idHabitacion":2,"fechaInicio":"2026-07-10","fechaFin":"2026-07-11","cantidadPersonas":2,"estadoReserva":"PENDIENTE"} {"idCliente":3,"idUsuario":2,"idHotel":1,"idHabitacion":3,"fechaInicio":"2026-07-12","fechaFin":"2026-07-13","cantidadPersonas":4,"estadoReserva":"PENDIENTE"} {"idCliente":4,"idUsuario":2,"idHotel":2,"idHabitacion":4,"fechaInicio":"2026-07-14","fechaFin":"2026-07-15","cantidadPersonas":3,"estadoReserva":"PENDIENTE"} {"idCliente":5,"idUsuario":1,"idHotel":2,"idHabitacion":5,"fechaInicio":"2026-07-16","fechaFin":"2026-07-18","cantidadPersonas":2,"estadoReserva":"PENDIENTE"}</pre>

ms-pago-service

Campo	Detalle
Estructura JSON esperada	<pre>{ "reservaId":1,"idUsuario":1,"monto":150000,"metodoPago":"TARJETA","estadoPago":"APROBADO","fechaPago":"2026-07-01T10:00:00"} </pre>
Reglas para poblar	- Debe existir reserva y usuario. - monto debe ser mayor a 0. - APROBADO confirma reserva; RECHAZADO/REEMBOLSADO cancelan; PENDIENTE deja pendiente.
Dependencias previas	Reserva y usuario existentes.
Orden correcto	Crear reserva antes de pago.
5 objetos recomendados	<pre>{ "reservaId":1,"idUsuario":1,"monto":150000,"metodoPago":"TARJETA","estadoPago":"APROBADO","fechaPago":"2026-07-01T10:00:00"} {"reservaId":2,"idUsuario":1,"monto":75000,"metodoPago":"TRANSFERENCIA","estadoPago":"PENDIENTE","fechaPago":"2026-07-01T11:00:00"} {"reservaId":3,"idUsuario":2,"monto":250000,"metodoPago":"DEBITO","estadoPago":"APROBADO","fechaPago":"2026-07-01T12:00:00"} {"reservaId":4,"idUsuario":2,"monto":100000,"metodoPago":"EFECTIVO","estadoPago":"RECHAZADO","fechaPago":"2026-07-01T13:00:00"} {"reservaId":5,"idUsuario":1,"monto":164000,"metodoPago":"TARJETA","estadoPago":"REEMBOLSADO","fechaPago":"2026-07-01T14:00:00"} </pre>

ms-notificacion-service

Campo	Detalle
Estructura JSON esperada	<pre>{ "idCliente":1,"idUsuario":1,"idReserva":1,"tipo":"EMAIL","mensaje":"Reserva creada","estado":"ENVIADA","fechaEnvio":"2026-07-01T10:00:00"} </pre>
Reglas para poblar	- Deben existir cliente, usuario y reserva. - No hay envio real ni catalogo de estados/tipos implementado.
Dependencias previas	Cliente, usuario y reserva.
Orden correcto	Crear despues de reserva.
5 objetos recomendados	<pre>{ "idCliente":1,"idUsuario":1,"idReserva":1,"tipo":"EMAIL","mensaje":"Reserva creada","estado":"ENVIADA","fechaEnvio":"2026-07-01T10:00:00"} {"idCliente":2,"idUsuario":1,"idReserva":2,"tipo":"SMS","mensaje":"Pago pendiente","estado":"PENDIENTE","fechaEnvio":"2026-07-01T11:00:00"} {"idCliente":3,"idUsuario":2,"idReserva":3,"tipo":"EMAIL","mensaje":"Pago aprobado","estado":"ENVIADA","fechaEnvio":"2026-07-01T12:00:00"} {"idCliente":4,"idUsuario":2,"idReserva":4,"tipo":"EMAIL","mensaje":"Reserva cancelada","estado":"ENVIADA","fechaEnvio":"2026-07-01T13:00:00"} {"idCliente":5,"idUsuario":1,"idReserva":5,"tipo":"PUSH","mensaje":"Check-in proximo","estado":"PENDIENTE","fechaEnvio":"2026-07-01T14:00:00"} </pre>

ms-resena-service

Campo	Detalle
Estructura JSON esperada	<pre>{ "idCliente":1,"idHotel":1,"idHabitacion":1,"idReserva":1,"calificacion":5,"comentario":"Excelente estadia","estadoResena":"PUBLICADA"} </pre>
Reglas para poblar	- Deben existir cliente, hotel y habitacion. - idReserva es obligatorio pero no se valida contra ms-reserva-service en el service actual. - calificacion entre 1 y 5.
Dependencias previas	Cliente, hotel, habitacion; reserva recomendada aunque no validada.
Orden correcto	Crear despues de reserva para mantener coherencia.

Campo	Detalle
5 objetos recomendados	<pre>{ "idCliente":1, "idHotel":1, "idHabitacion":1, "idReserva":1, "calificacion":5, "comentario": "Excelente estadia", "estadoResena": "PUBLICADA" }, { "idCliente":2, "idHotel":1, "idHabitacion":2, "idReserva":2, "calificacion":4, "comentario": "Muy buena atencion", "estadoResena": "PUBLICADA" }, { "idCliente":3, "idHotel":1, "idHabitacion":3, "idReserva":3, "calificacion":3, "comentario": "Correcto", "estadoResena": "PUBLICADA" }, { "idCliente":4, "idHotel":2, "idHabitacion":4, "idReserva":4, "calificacion":2, "comentario": "Puede mejorar", "estadoResena": "PENDIENTE" }, { "idCliente":5, "idHotel":2, "idHabitacion":5, "idReserva":5, "calificacion":5, "comentario": "Volveria", "estadoResena": "PUBLICADA" }</pre>

ms-servicio-adicional-service

Campo	Detalle
Estructura JSON esperada	<pre>{ "idHotel":1, "idReserva":1, "nombre": "Spa", "descripcion": "Acceso a spa", "precio":25000, "estado": "ACTIVO" }</pre>
Reglas para poblar	- Debe existir hotel. - Si se envia idReserva, debe existir reserva. - precio positivo es recomendacion; no esta validado actualmente.
Dependencias previas	Hotel obligatorio; reserva opcional.
Orden correcto	Crear despues de hotel; si se asocia a reserva, crear despues de reserva.
5 objetos recomendados	<pre>{ "idHotel":1, "idReserva":1, "nombre": "Desayuno", "descripcion": "Desayuno buffet", "precio":12000, "estado": "ACTIVO" }, { "idHotel":1, "idReserva":2, "nombre": "Spa", "descripcion": "Acceso a spa", "precio":25000, "estado": "ACTIVO" }, { "idHotel":1, "idReserva":null, "nombre": "Estacionamiento", "descripcion": "Estacionamiento diario", "precio":8000, "estado": "ACTIVO" }, { "idHotel":2, "idReserva":4, "nombre": "Tour", "descripcion": "Tour por la ciudad", "precio":35000, "estado": "ACTIVO" }, { "idHotel":2, "idReserva":5, "nombre": "Late checkout", "descripcion": "Salida extendida", "precio":20000, "estado": "ACTIVO" }</pre>

Matriz De Pruebas Manuales

Caso	Datos validos	Datos invalidos	Resultado esperado	HTTP	Revisar en Swagger	Revisar en BD
ms-auth-usuarios-service - crear/listar	{"nombre":"admin","password":"admin123","rol":"ADMIN"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-auth-usuarios-service - buscar inexistente	GET /api/v1/auth/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-auth-usuarios-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-auth-usuarios-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-cliente-service - crear/listar	{"rutDocumento":"11111111-1","telefono":"+56911111111","direccion":"Av. Siempre Viva 123","rolCliente":"TITULAR","tipoCliente":"NACIONAL"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-cliente-service - buscar inexistente	GET /api/v1/clientes/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-cliente-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-cliente-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-hotel-service - crear/listar	{"nombre":"Hotel Transilvania","direccion":"Camino del Bosque 1","ciudad":"Santiago","pais":"Chile","categoria":"5 estrellas","descripcion":"Hotel tematico"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-hotel-service - buscar inexistente	GET /api/v1/hoteles/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-hotel-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-hotel-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-habitacion-service - crear/listar	{"numeroHabitacion":"101","tipoHabitacion":"DOBLE","precioBase":75000,"capacidad":2,"estadoHabitacion":"DISPONIBLE","idHotel":1}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-habitacion-service - buscar inexistente	GET /api/v1/habitaciones/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-habitacion-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-habitacion-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-disponibilidad-service - crear/listar	{"idHabitacion":1,"fecha":"2026-07-10","estado":"DISPONIBLE"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-disponibilidad-service - buscar inexistente	GET /api/v1/disponibilidades/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-disponibilidad-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios

Caso	Datos validos	Datos invalidos	Resultado esperado	HTTP	Revisar en Swagger	Revisar en BD
ms-disponibilidad-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-reserva-service - crear/listar	{"idCliente":1,"idUsuario":1,"idHotel":1,"idHabitacion":1,"fechaInicio":"2026-07-10","fechaFin":"2026-07-12","cantidadPersonas":2,"estadoReserva":"PENDIENTE"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-reserva-service - buscar inexistente	GET /api/v1/reservas/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-reserva-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-reserva-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-pago-service - crear/listar	{"reservald":1,"idUsuario":1,"monto":150000,"metodoPago":"TARJETA","estadoPago":"APROBADO","fechaPago":"2026-07-01T10:00:00"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-pago-service - buscar inexistente	GET /api/v1/pagos/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-pago-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-pago-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-notificacion-service - crear/listar	{"idCliente":1,"idUsuario":1,"idReserva":1,"tipo":"RESERVA","mensaje":"Reserva creada","estado":"PENDIENTE","fechaEnvio":"2026-07-01T10:00:00"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-notificacion-service - buscar inexistente	GET /api/v1/notificaciones/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-notificacion-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-notificacion-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-resena-service - crear/listar	{"idCliente":1,"idHotel":1,"idHabitacion":1,"idReserva":1,"calificacion":5,"comentario":"Excelente estadia","estadoResena":"PUBLICADA"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-resena-service - buscar inexistente	GET /api/v1/resenas/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros
ms-resena-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-resena-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios
ms-servicio-adicional-service - crear/listar	{"idHotel":1,"idReserva":1,"nombre":"Spa","descripcion":"Acceso a spa","precio":25000,"estado":"ACTIVO"}	Campos obligatorios vacios o null	Recurso creado o lista visible	200/201 o 400	Status, body, mensajes de validacion	Registro creado/actualizado
ms-servicio-adicional-service - buscar inexistente	GET /api/v1/servicios-adicionales/999999	ID inexistente	Mensaje de no encontrado	404	Body con timestamp/status/message	No debe crear registros

Caso	Datos validos	Datos invalidos	Resultado esperado	HTTP	Revisar en Swagger	Revisar en BD
ms-servicio-adicional-service - sin token	Endpoint protegido sin Authorization	Token ausente	Bloqueo de seguridad	401	Respuesta Unauthorized	Sin cambios
ms-servicio-adicional-service - rol incorrecto	Token USER en endpoint ADMIN	Rol insuficiente	Acceso denegado	403	Mensaje de permisos	Sin cambios

Errores Esperados

HTTP	Causa típica	Ejemplo para provocar	Respuesta esperada
400 Bad Request	Validacion Bean Validation, JSON invalido o IllegalArgumentException	Enviar campo obligatorio vacio, estado invalido o monto <= 0	Map con timestamp/status y message o errors
401 Unauthorized	Sin token o token invalido	Ejecutar endpoint protegido sin Authorize	Respuesta de Spring Security
403 Forbidden	Token valido con rol no permitido	USER intentando DELETE /api/v1/hoteles/{id}	Mensaje de permisos si lo captura GlobalExceptionHandler
404 Not Found	NoSuchElementException al buscar id inexistente	GET /api/v1/reservas/999999	Map con status 404 y message
409 Conflict	No esta mapeado explicitamente en los handlers revisados	Duplicar disponibilidad devuelve actualmente 400 por IllegalArgumentException	Recomendacion: mapear duplicados a 409
500 Internal Server Error	Excepcion no controlada	Falla inesperada de codigo	Map con status 500 y mensaje generico
503 Service Unavailable	WebClientRequestException en algunos servicios	Apagar ms-hotel-service y crear habitacion	Map con status 503 si el handler del servicio lo contempla

Pruebas Negativas Recomendadas

- Crear con campos vacios: validar mensajes @NotBlank y @NotNull.
- Crear con ids inexistentes: hotel/cliente/usuario/habitacion/reserva segun dependencia.
- Crear reserva con fechaInicio igual o posterior a fechaFin: debe devolver 400.
- Crear reserva CONFIRMADA sin disponibilidad: debe devolver 400/404 segun falte registro o estado no disponible.
- Crear disponibilidad duplicada misma habitacion y fecha: actualmente 400; recomendado 409.
- Crear habitacion con estado INVALIDO: debe devolver 400.
- Crear pago con monto 0 o negativo: debe devolver 400.
- Crear pago APROBADO: revisar que reserva pase a CONFIRMADA y disponibilidad a OCUPADA.
- Crear pago RECHAZADO: revisar que reserva quede CANCELADA.
- Probar microservicio dependiente apagado: esperar 503 si existe handler WebClientRequestException; si no, puede caer en 500.

Notas De Implementacion Detectadas

- No hay endpoints PATCH en los controllers revisados.
- Reserva no fuerza estado PENDIENTE al crear; si el body envia CONFIRMADA, intenta confirmar y ocupar disponibilidad. La regla solicitada de 'al crear debe quedar PENDIENTE' no esta implementada actualmente.
- Pago no impide dos pagos para la misma reserva. Recomendacion: validar existencia de pago por reserva o crear indice unico si la regla aplica.
- Servicio adicional y notificacion no tienen catalogo de estados implementado.
- Resena exige idReserva pero no valida la reserva por WebClient.
- Varios duplicados o conflictos se devuelven como 400 por IllegalArgumentException; si se quiere 409, se debe agregar excepcion especifica o ResponseStatusException(CONFLICT).